Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №6
По дисциплине «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Парфенюк Д. П.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы: освоить приемы тестирования кода на примере использования пакета pytest

Задание 1:

8) Напишите метод int levenshteinDistance(String s, String t) рассчитывающий расстояние Ливенштейна для двух строк. Расстояние Ливенштейна между двумя строками – это то количество посимвольных трансформаций необходимое, что бы превратить одну строку в другую.

Shopping.py
Решение:

```
"""Модуль для управления корзиной покупок и обработки купонов."""
import requests


class Cart:
    """Класс, представляющий корзину товаров."""

    def __init__(self):
        """Инициализация пустой корзины."""
        self.items = []

    def add_item(self, name, price):
        """Добавление товара в корзину."""
        if price < 0:
            raise ValueError("Цена не может быть отрицательной")
        self.items.append({"name": name, "price": price})

    def total(self):
        """Расчет общей стоимости товаров в корзине."""
        return sum(item["price"] for item in self.items)

    def apply_discount(self, percent):
        """Применение процентной скидки ко всем товарам."""
        if not 0 <= percent <= 100:
            raise ValueError("Недопустимая скидка")
        for item in self.items:
            item["price"] *= 1 - percent / 100


def log_purchase(item):
    """Логирование покупки во внешнюю систему."""
    response = requests.post("https://example.com/log", json=item, timeout=5)
    return response.status_code


def apply_coupon(cart, coupon_code):
    """Применение купона к корзине."""
    coupons = {"SAVE10": 10, "HALF": 50}
    if coupon_code in coupons:
        cart.apply_discount(coupons[coupon_code])
    else:
        raise ValueError("Invalid coupon")
```

String_utils.py
Решение:

```
"""Модуль с утилитами для работы со строками."""


def levenshtein_distance(s, t):
    """
```

```
    Вычисляет расстояние Левенштейна между двумя строками.
    """
    if s is None and t is None:
        raise TypeError("Оба аргумента None")
    if s is None or t is None:
        return -1

    rows = len(s) + 1
    cols = len(t) + 1
    dist = [[0 for _ in range(cols)] for _ in range(rows)]

    for i in range(1, rows):
        dist[i][0] = i
    for i in range(1, cols):
        dist[0][i] = i

    for col in range(1, cols):
        for row in range(1, rows):
            if s[row - 1] == t[col - 1]:
                cost = 0
            else:
                cost = 1
            dist[row][col] = min(
                dist[row - 1][col] + 1, dist[row][col - 1] + 1, dist[row -
1][col - 1] + cost
            )

    return dist[rows - 1][cols - 1]
```

Test_lab6.py
Решение:

```
"""Тестовый модуль для проверки корзины и строковых утилит."""
# pylint: disable=missing-function-docstring, redefined-outer-name, invalid-
name
from unittest.mock import patch
import pytest
from shopping import Cart, log_purchase, apply_coupon
from string_utils import levenshtein_distance


@pytest.fixture
def empty_cart():
    """Фикстура для создания пустой корзины."""
    return Cart()


def test_add_item(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    assert len(empty_cart.items) == 1


def test_negative_price(empty_cart):
    with pytest.raises(ValueError):
        empty_cart.add_item("Negative", -5.0)


def test_total(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 20.0)
    assert empty_cart.total() == 30.0
```

```python
@pytest.mark.parametrize(
    "percent, expected_total",
    [
        (0, 100.0),
        (50, 50.0),
        (100, 0.0),
    ],
)
def test_apply_discount(empty_cart, percent, expected_total):
    empty_cart.add_item("Item", 100.0)
    empty_cart.apply_discount(percent)
    assert empty_cart.total() == expected_total


@pytest.mark.parametrize("percent", [-10, 110])
def test_invalid_discount(empty_cart, percent):
    empty_cart.add_item("Item", 100.0)
    with pytest.raises(ValueError):
        empty_cart.apply_discount(percent)


def test_log_purchase():
    item = {"name": "Apple", "price": 10.0}
    with patch("requests.post") as mocked_post:
        mocked_post.return_value.status_code = 200
        status = log_purchase(item)
        mocked_post.assert_called_once_with("https://example.com/log",
json=item, timeout=5)
        assert status == 200


def test_apply_coupon(empty_cart):
    empty_cart.add_item("Item", 100.0)
    apply_coupon(empty_cart, "SAVE10")
    assert empty_cart.total() == 90.0


@pytest.mark.parametrize(
    "s, t, expected",
    [
        ("", "", 0),
        ("", "a", 1),
        ("aaapppp", "", 7),
        ("frog", "fog", 1),
        ("fly", "ant", 3),
        ("elephant", "hippo", 7),
        ("hippo", "elephant", 7),
        ("hippo", "zzzzzzzz", 8),
        ("hello", "hallo", 1),
        (None, "a", -1),
        ("a", None, -1),
    ],
)
def test_levenshtein_values(s, t, expected):
    assert levenshtein_distance(s, t) == expected


def test_levenshtein_type_error():
    with pytest.raises(TypeError):
        levenshtein_distance(None, None)
```

```
PS C:\Users\asus home\PycharmProjects\spp_po12\reports\Parfenyuk_D_P\lab6spp\src> pytest test_lab6.py
========================================================================================== test session starts
platform win32 -- Python 3.13.1, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\asus home\PycharmProjects\spp_po12\reports\Parfenyuk_D_P\lab6spp\src
plugins: anyio-4.13.0
collected 22 items

test_lab6.py ......................

========================================================================================== 22 passed in 0.15s
PS C:\Users\asus home\PycharmProjects\spp_po12\reports\Parfenyuk_D_P\lab6spp\src>
```

Вывод: освоил приемы тестирования кода на примере использования пакета pytest